

CHAPTER 1, 3, 4, 5

Scientific = Fortran - large number float array

Business = COBOL - report, use decimal

Artificial = LISP - linked symbol

System prog = efficient C

web software = HTML, PHP

Readability - Writability - Reliability - Cost

Computer architecture - Von Neumann

Data and program stored memory

memory separate (CPU) → Control unit
Arithmetic logic unit

Imperative (C, mlk) = AUL Opp language

functional = LPS, scheme, ml

Logic = Prolog (RULE)

markup = JSL, XSLT

Reliability ↑ cost ↑

writability ↑ readability ↓

writability ↑ readability ↓

C++ pointer

Implementation method

- Compilation $\rightarrow$ large application $\rightarrow$ fast exec ^{slow transk}
- Pure Interpretation $\rightarrow$ small program $\rightarrow$ ^{slow transk} fast execution ^{slow execution}
- $\rightarrow$ Hybrid System $\rightarrow$ faster

Compilation = lexical $\rightarrow$ syntax $\rightarrow$ semantic $\rightarrow$ code generation

JUST IN TIME $\Rightarrow$ Java, .Net

Syntax $\Rightarrow$ form lexical tree (int a)

Semantic = orta kod (int a onlon)

Lexical = Karakter (sum, int)

Token = terminalna (semicolon)

JOhn Backus syntax ALgo 58

$\hookrightarrow$ context free grammar

Describe the syntax $\rightarrow$ language generation

Ambiguity = 2 farklı olon

extended BNF = $\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle \{ + | - \} \langle \text{term} \rangle$

BNF $\Rightarrow \langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle$

parse tree $\Rightarrow$ A hierarchical representation derivation

Compile $\Rightarrow$ Code Gen

Load = static memory

Run time = non static non variable

Static binding = unchanged program execution

Dynamic binding change program execution

SCOPE = Static, dynamic,

Pair wise disjointness test = non left recursive

Reserved word = didn't identify grammar before but us...

Left recursive

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

$E' \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T' \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

Left derivation bbacaaabb

$\langle S \rangle \rightarrow \langle A \rangle c \langle B \rangle b$

$\langle A \rangle \rightarrow b \langle A \rangle \mid a$

$\langle B \rangle \rightarrow a \langle B \rangle \mid b$

$\langle S \rangle \rightarrow \langle A \rangle c \langle B \rangle b$

$b \langle A \rangle c \langle B \rangle b$

$bb \langle A \rangle c \langle B \rangle b$

$bb a c \langle B \rangle b$

$bb a c a a \langle B \rangle b$

$bb a c a a b b$

left recursive

$$\langle A \rangle \rightarrow \langle A \rangle * \langle B \rangle \mid \langle A \rangle / \langle B \rangle \mid a$$

$$\langle B \rangle \rightarrow \langle B \rangle + \langle C \rangle \mid \langle B \rangle - \langle C \rangle \mid b$$

$$\langle C \rangle \rightarrow c$$

$$\langle A' \rangle \rightarrow a \langle A' \rangle$$

$$\langle A' \rangle \rightarrow * \langle B \rangle \langle A' \rangle \mid / \langle B \rangle \langle A' \rangle \mid \epsilon$$

$$\langle B' \rangle \rightarrow b \langle B' \rangle$$

$$\langle B' \rangle \rightarrow + \langle C \rangle \langle B' \rangle \mid - \langle C \rangle \langle B' \rangle \mid \epsilon$$

$$\langle C \rangle \rightarrow c$$

Static scope	Life times
Code sequentially compiled	global = static
Dynamic scope	Local = stack dynamic
Compile sequentially	pointer = explicit heap dynamic

